.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica For The Win

12 de octubre de 2026

Guadalupe Schell
114335

Abril Erin Borsnich
114760

Índice

1. Análisis del algoritmo **3**
1.1. Enunciado del problema . . . 3
1.2. Análisis del problema y algoritmo planteado . . . 3
1.3. Ejemplos de ejecución . . . 4
1.3.1. Ejemplo 1 . . . 4
1.3.2. Ejemplo 2: Sophia no puede ganar . . . 5
1.3.3. Ejemplo 3: estrategia greedy frente a programación dinámica . . . 5

2. Optimalidad del algoritmo **6**
2.1. Demostración . . . 6

3. Implementación del algoritmo **7**
3.1. Código funcional . . . 7
3.2. Complejidad . . . 8
3.2.1. Complejidad del algoritmo de Programación Dinámica . . . 9
3.2.2. Complejidad de la reconstrucción . . . 9
3.2.3. Conclusión de costo global . . . 9

4. Análisis experimental **10**
4.1. Variabilidad de los valores de las monedas . . . 10
4.2. Medición de tiempos y análisis de complejidad . . . 10

5. Conclusiones **11**

1. Análisis del algoritmo

1.1. Enunciado del problema

Sophia y Mateo juegan a un juego utilizando una fila de n monedas, donde cada moneda tiene asociado un valor. En cada turno solamente puede retirarse una de las dos monedas que se encuentran en los extremos de la fila. Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Ganará quien acumule el mayor valor total.

Mateo aprendió sobre algoritmos greedy, y aplica esa técnica a la hora de elegir una moneda. En cada turno elige la moneda de mayor valor entre los dos extremos disponibles.

Sophia, en cambio, comenzó a aprender sobre programación dinámica y va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

El objetivo es determinar la máxima ganancia que puede obtener Sophia si toma sus decisiones de manera óptima, teniendo en cuenta que Sophia empieza el juego y que Mateo siempre elegirá la moneda más grande para sí entre la primera y la última moneda. Esto no necesariamente le asegurará a Sophia ganar, ya que la estrategia de Mateo puede impedirlo.

1.2. Análisis del problema y algoritmo planteado

Sea M el arreglo de monedas, donde $M[i]$ representa el valor de la moneda ubicada en la posición i. Definimos

$OPT[i][j]$ = máxima ganancia que puede obtener Sophia si le toca jugar y quedan las monedas $[i, j]$.

La definición se aplica a intervalos no vacíos con $i \leq j$. Si $i > j$, el intervalo está vacío y su ganancia es cero.

En cada estado, Sophia tiene dos alternativas.

- **Tomar la moneda izquierda $M[i]$.** Después de esta elección, Mateo debe elegir entre $M[i+1]$ y $M[j]$.
  - Si $M[i+1] \geq M[j]$, Mateo toma la moneda $M[i+1]$ y Sophia vuelve a jugar con el intervalo $[i+2, j]$.
  - Si $M[i+1] < M[j]$, Mateo toma la moneda $M[j]$ y Sophia vuelve a jugar con el intervalo $[i+1, j-1]$.
- **Tomar la moneda derecha $M[j]$.** Después de esta elección, Mateo debe elegir entre $M[i]$ y $M[j-1]$.
  - Si $M[i] \geq M[j-1]$, Mateo toma la moneda $M[i]$ y Sophia vuelve a jugar con el intervalo $[i+1, j-1]$.
  - Si $M[i] < M[j-1]$, Mateo toma la moneda $M[j-1]$ y Sophia vuelve a jugar con el intervalo $[i, j-2]$.

Por lo tanto, Sophia compara el valor que puede asegurarse al tomar cada uno de los extremos, descontando la moneda que Mateo elegirá inmediatamente después. Definimos las contribuciones futuras de cada alternativa como

$$A(i,j) = \begin{cases} OPT[i+2][j], & \text{si } M[i+1] \geq M[j], \\ OPT[i+1][j-1], & \text{si } M[i+1] < M[j], \end{cases}$$

y

$$B(i,j) = \begin{cases} OPT[i+1][j-1], & \text{si } M[i] \geq M[j-1], \\ OPT[i][j-2], & \text{si } M[i] < M[j-1]. \end{cases}$$

La recurrencia es entonces

$$OPT[i][j] = \max\{M[i] + A(i,j),\ M[j] + B(i,j)\}.$$

Los casos base son:

- Si $i > j$, entonces $OPT[i][j] = 0$, porque no quedan monedas.
- Si $i = j$, entonces $OPT[i][i] = M[i]$, porque Sophia toma la única moneda disponible cuando le toca jugar.

En la implementación, los subintervalos vacíos se tratan como si tuvieran valor cero. La recurrencia se evalúa para intervalos de longitud creciente: primero los de longitud uno, luego los de longitud dos y así sucesivamente hasta el intervalo completo $[0, n-1]$. La respuesta buscada queda en $OPT[0][n-1]$.

Como la decisión de Mateo queda determinada por los extremos disponibles y por su regla greedy, cada alternativa de Sophia conduce a un único subproblema. De esta manera, basta con comparar las dos alternativas y guardar el máximo, sin recalcular repetidamente los mismos subproblemas.

1.3. Ejemplos de ejecución

1.3.1. Ejemplo 1

Consideremos la secuencia de seis monedas

$$M = [20, 5, 15, 10, 8, 25].$$

La matriz OPT almacena en cada celda la máxima ganancia que puede obtener Sophia cuando le toca jugar y quedan las monedas del intervalo indicado. Los intervalos vacíos no se almacenan en la matriz y se consideran de valor cero.

Evaluación de las dos primeras alternativas. Al evaluar el intervalo completo $[0, 5]$, Sophia tiene dos opciones:

- **Tomar 20, el extremo izquierdo.** Queda $[5, 15, 10, 8, 25]$. Mateo compara 5 y 25 y toma 25. Sophia vuelve a jugar con $[5, 15, 10, 8]$, cuyo valor es $OPT[1][4] = 23$. La ganancia total que puede obtener Sophia por esta alternativa es $20 + 23 = 43$.
- **Tomar 25, el extremo derecho.** Queda $[20, 5, 15, 10, 8]$. Mateo compara 20 y 8 y toma 20. Sophia vuelve a jugar con $[5, 15, 10, 8]$, cuyo valor es $OPT[1][4] = 23$. La ganancia total de Sophia por esta alternativa es $25 + 23 = 48$.

Por lo tanto, la decisión óptima inicial es tomar la moneda derecha, de valor 25.

La matriz final es:

$i \setminus j$	0	1	2	3	4	5
0	20	20	25	35	35	**48**
1	–	5	**15**	15	**23**	40
2	–	–	15	15	23	35
3	–	–	–	10	10	33
4	–	–	–	–	8	25
5	–	–	–	–	–	25

Reconstrucción del camino óptimo. La secuencia de decisiones es:

1. El estado inicial es $OPT[0][5] = 48$. Sophia toma la moneda derecha, de valor 25. Mateo compara los extremos restantes, 20 y 8, y toma 20.
2. Queda el intervalo $[1, 4]$, correspondiente a $[5, 15, 10, 8]$, con $OPT[1][4] = 23$. Sophia toma la moneda derecha, de valor 8. Mateo compara 5 y 10 y toma 10.
3. Queda el intervalo $[1, 2]$, correspondiente a $[5, 15]$, con $OPT[1][2] = 15$. Sophia toma 15. Mateo recibe la moneda restante, de valor 5.

Sophia obtiene $25 + 8 + 15 = 48$, mientras que Mateo obtiene $20 + 10 + 5 = 35$. Por lo tanto, en este ejemplo Sophia gana.

1.3.2. Ejemplo 2: Sophia no puede ganar

Consideremos ahora

$$M = [1, 10, 5].$$

Al evaluar el intervalo completo $[0, 2]$, Sophia compara las dos alternativas:

- **Tomar 1, el extremo izquierdo.** Mateo recibe el intervalo $[10, 5]$ y toma 10. Sophia se queda con la moneda de valor 5. Su ganancia es $1 + 5 = 6$.
- **Tomar 5, el extremo derecho.** Mateo recibe el intervalo $[1, 10]$ y toma 10. Sophia se queda con la moneda de valor 1. Su ganancia es $5 + 1 = 6$.

La matriz correspondiente es:

$i \setminus j$	0	1	2
0	1	10	**6**
1	–	10	**10**
2	–	–	5

Ambas alternativas dan una ganancia de 6 para Sophia. Si se desempata eligiendo el extremo izquierdo, Sophia toma 1, Mateo toma 10 y Sophia toma 5. Sophia obtiene 6 y Mateo obtiene 10. Por lo tanto, la ganancia óptima de Sophia es 6, pero no alcanza para ganar. Este ejemplo muestra que el algoritmo maximiza la ganancia de Sophia, aunque las reglas del juego no siempre le permiten superar a Mateo.

1.3.3. Ejemplo 3: estrategia greedy frente a programación dinámica

Consideremos la secuencia

$$M = [1, 2, 1, 1, 1].$$

Este ejemplo permite comparar una estrategia greedy para Sophia con la estrategia óptima calculada mediante programación dinámica. Para que la comparación sea reproducible, suponemos que, ante un empate entre los extremos, tanto Sophia como Mateo eligen el extremo izquierdo.

Si Sophia también juega greedy.

1. Sophia ve 1 y 1 en los extremos y, por el desempate, toma el 1 de la izquierda. Queda $[2, 1, 1, 1]$.
2. Mateo compara 2 y 1 y toma el 2 de la izquierda. Queda $[1, 1, 1]$.
3. Sophia toma el 1 de la izquierda. Queda $[1, 1]$.
4. Mateo toma el 1 de la izquierda y Sophia recibe el último 1.

Sophia suma $1 + 1 + 1 = 3$ y Mateo suma $2 + 1 = 3$. El resultado es un empate.

Si Sophia juega de forma óptima.

1. Sophia toma el 1 del extremo derecho. Queda $[1, 2, 1, 1]$.
2. Mateo encuentra un empate entre los extremos, ambos de valor 1, y toma el de la izquierda. Queda $[2, 1, 1]$.
3. Sophia toma el 2 de la izquierda. Queda $[1, 1]$.
4. Mateo toma el 1 de la izquierda y Sophia recibe el último 1.

En este caso, Sophia suma $1 + 2 + 1 = 4$ y Mateo suma $1 + 1 = 2$. Sophia gana.

La diferencia se debe a que la primera elección de Sophia no se basa únicamente en el valor inmediato de la moneda. Aunque ambos extremos valen 1, tomar el extremo derecho cambia la respuesta greedy de Mateo: este toma el 1 izquierdo y deja disponible la moneda de valor 2 para el siguiente turno de Sophia. La programación dinámica detecta esta consecuencia futura y obtiene una ganancia de 4, superior a los 3 puntos que consigue la estrategia greedy.

2. Optimalidad del algoritmo

2.1. Demostración

Demostraremos por inducción sobre la longitud del intervalo $l = j - i + 1$ que $OPT[i][j]$ representa la máxima ganancia que puede obtener Sophia cuando le toca jugar y quedan las monedas del intervalo $[i, j]$, suponiendo que Mateo aplica siempre la estrategia greedy especificada.

Casos base. Si el intervalo está vacío ($i > j$), no quedan monedas y la ganancia de Sophia es cero. Por eso, $OPT[i][j] = 0$ para los intervalos vacíos.

Si el intervalo contiene una sola moneda ($i = j$), Sophia está obligada a tomarla cuando le toca jugar. Su ganancia es exactamente $M[i]$, por lo que $OPT[i][i] = M[i]$.

Ambos casos base son correctos.

Hipótesis inductiva. Supongamos que para todo intervalo de longitud estrictamente menor que k, la celda correspondiente de OPT almacena correctamente la máxima ganancia que Sophia puede obtener desde ese estado.

Paso inductivo. Consideremos un intervalo $[i, j]$ de longitud k. Sophia tiene dos alternativas posibles.

- **Tomar la izquierda** $M[i]$**.** Después de esa elección, Mateo debe elegir entre $M[i + 1]$ y $M[j]$. Su estrategia determina cuál de las dos monedas retira. Si toma $M[i + 1]$, el siguiente intervalo para Sophia es $[i + 2, j]$; en caso contrario, es $[i + 1, j - 1]$. Ambos intervalos tienen longitud menor que k. Por hipótesis inductiva, sus valores en OPT son correctos. Por lo tanto, la mejor ganancia asociada a esta alternativa es $M[i] + A(i, j)$.
- **Tomar la derecha** $M[j]$**.** Después de esa elección, Mateo elige entre $M[i]$ y $M[j - 1]$. Según cuál tome, el intervalo posterior para Sophia será $[i + 1, j - 1]$ o $[i, j - 2]$. También son intervalos de longitud menor que k, por lo que la hipótesis inductiva garantiza que sus valores son óptimos. La ganancia asociada a esta alternativa es $M[j] + B(i, j)$.

Sophia conoce la regla de Mateo y, por lo tanto, puede anticipar la respuesta de este en cada alternativa. Como las únicas primeras jugadas posibles son tomar el extremo izquierdo o el derecho, la mejor ganancia que puede asegurarse es el máximo de ambas alternativas:

$$OPT[i][j] = \text{máx}\{M[i] + A(i, j),\ M[j] + B(i, j)\}.$$

En consecuencia, $OPT[i][j]$ es correcto para todo intervalo de longitud k. Por inducción, todos los estados de la matriz se calculan correctamente. En particular, $OPT[0][n-1]$ contiene la máxima ganancia que Sophia puede obtener en el juego completo. Queda demostrada la optimalidad del algoritmo.

3. Implementación del algoritmo

3.1. Código funcional

La implementación utiliza un enfoque iterativo (*Bottom-Up*). Se construye la matriz OPT desde los intervalos más pequeños hasta el intervalo completo.

```python
def resolver(monedas):
    n = len(monedas)
    if n == 0:
        return [], 0, 0


    opt = [[0] * n for _ in range(n)]

    # Caso base queda una sola moneda
    for i in range(n):
        opt[i][i] = monedas[i]

    for largo in range(2, n + 1):
        for i in range(n - largo + 1):
            j = i + largo - 1

            # sophia toma la moneda izquierda
            if monedas[i + 1] >= monedas[j]:
                if i + 2 <= j:
                    futuro_izq = opt[i + 2][j]
                else:
                    futuro_izq = 0
            else:
                if i + 1 <= j - 1:
                    futuro_izq = opt[i + 1][j - 1]
                else:
                    futuro_izq = 0
            izq = monedas[i] + futuro_izq

            # sophia toma la moneda derecha
            if monedas[i] >= monedas[j - 1]:
                if i + 1 <= j - 1:
                    futuro_der = opt[i + 1][j - 1]
                else:
                    futuro_der = 0
            else:
                if i <= j - 2:
                    futuro_der = opt[i][j - 2]
                else:
                    futuro_der = 0
            der = monedas[j] + futuro_der

            # en caso de empate se elige el extremo izquierdo
            if izq >= der:
                opt[i][j] = izq
            else:
                opt[i][j] = der

    return reconstruir_elecciones(opt, monedas)

def reconstruir_elecciones(opt, monedas):
    n = len(monedas)
    decisiones = []
    i = 0
    j = n - 1
```

```
56
57      ganancia_mateo = 0
58
59      while i <= j:
60          if i == j:
61              # Caso base: queda unicamente esa moneda
62              decisiones.append(f"Sophia toma la izquierda ({monedas[i]})")
63              break
64
65          # uso alternativa de tomar la izquierda para deducir la decisi n
66          if monedas[i + 1] >= monedas[j]:
67              if i + 2 <= j:
68                  futuro_izq = opt[i + 2][j]
69              else:
70                  futuro_izq = 0
71          else:
72              if i + 1 <= j - 1:
73                  futuro_izq = opt[i + 1][j - 1]
74              else:
75                  futuro_izq = 0
76          valor_si_izq = monedas[i] + futuro_izq
77
78          # Si el valor calculado coincide con la matriz, Sophia eligio la izquierda
79          if opt[i][j] == valor_si_izq:
80              decisiones.append(f"Sophia toma la izquierda ({monedas[i]})")
81              i += 1
82          else:
83              decisiones.append(f"Sophia toma la derecha ({monedas[j]})")
84              j -= 1
85
86          # Turno de Mateo (Estrategia Greedy)
87          if i <= j:
88              if monedas[i] >= monedas[j]:
89                  decisiones.append(f"Mateo toma la izquierda ({monedas[i]})")
90                  ganancia_mateo += monedas[i]
91                  i += 1
92              else:
93                  decisiones.append(f"Mateo toma la derecha ({monedas[j]})")
94                  ganancia_mateo += monedas[j]
95                  j -= 1
96
97      return decisiones, opt[0][n-1], ganancia_mateo
```

La función `resolver(monedas)` recibe el arreglo con los valores de las monedas. Primero crea la matriz `opt` que almacena la ganancia máxima de Sophia para cada intervalo.

Luego se inicializan los casos base y se completan los intervalos por longitud creciente. Para cada intervalo se calculan las ganancias posibles si Sophia toma la moneda izquierda o la derecha, considerando en cada caso la elección Greedy de Mateo. Se almacena el mayor de los dos valores como el óptimo de dicha posición.

Finalmente, se retorna la función `reconstruir_elecciones(opt, monedas)` que se encarga de reconstruir la secuencia de jugadas calculando el valor que tendría opt[i][j] si Sophia hubiera elegido la izquierda y deduciendo así la verdadera elección. Mientras tanto, las elecciones de Mateo se deciden siguiendo la estrategia *Greedy*. La función devuelve la lista de decisiones, la ganancia máxima de Sophia y la ganancia total de Mateo.

3.2. Complejidad

Analizaremos por separado la complejidad temporal y espacial del cálculo de la matriz de programación dinámica y la complejidad de reconstruir la secuencia de decisiones.

3.2.1. Complejidad del algoritmo de Programación Dinámica

Sea n la cantidad de monedas de la fila inicial. La matriz `opt` tiene un tamaño de $n \times n$. Como solo calculamos los casos donde el inicio del intervalo no supera al final ($i \leq j$), solo estamos llenando la mitad superior de la matriz. Esto significa que la cantidad de estados que el algoritmo procesa es, aproximadamente, la mitad de n^2, es decir, $\mathcal{O}(\frac{n^2}{2})$. Como en notación de orden, las constantes se descartan, nos queda que el algoritmo calcula $\mathcal{O}(n^2)$ estados.

El algoritmo recorre las longitudes de los intervalos desde 2 hasta n. Para cada longitud, se exploran todos los intervalos posibles. En cada estado, se realiza un número constante de comparaciones lógicas, sumas aritméticas y accesos de lectura/escritura a la matriz, resultando en operaciones de costo $\mathcal{O}(1)$. Por lo tanto, el tiempo total del algoritmo estará determinado únicamente por la cantidad de celdas que debemos llenar. Como la matriz de subproblemas tiene una dimensión de $n \times n$, el costo de completar la matriz da como resultado una complejidad temporal total de $\mathcal{O}(n^2)$.

La inicialización de la matriz también requiere un tiempo de $\mathcal{O}(n^2)$, por lo que el orden del costo temporal general no cambia.

En consecuencia, la complejidad temporal del algoritmo de programación dinámica es:

$$\boxed{\mathcal{O}(n^2)}$$

y su complejidad espacial adicional es:

$$\boxed{\mathcal{O}(n^2)}$$

3.2.2. Complejidad de la reconstrucción

La reconstrucción de las jugadas empieza desde la solución completa en la esquina superior de la matriz, donde $i = 0$ y $j = n - 1$. A partir de ahí, el algoritmo avanza mediante un bucle que simula la partida desde el estado inicial. En el turno de Sophia, calculamos el valor que se obtendría si ella hubiera elegido la moneda izquierda y lo comparamos con el resultado real guardado en la celda $OPT[i][j]$. Si ambos números coinciden, confirmamos que tomó la izquierda, de lo contrario, deducimos que eligió la derecha. Luego de mover el índice correspondiente, simulamos el turno de Mateo eliminando la moneda más grande según su lógica *Greedy*.

Como cada una de estas comprobaciones matemáticas internas se resuelve en un tiempo constante $\mathcal{O}(1)$ y en cada iteración del bucle se descartan monedas de la mesa, el ciclo se ejecuta a lo sumo n veces hasta vaciar el tablero. Esto hace que el costo de reconstruir todo el camino sea lineal, es decir, de $\mathcal{O}(n)$.

En cuanto al espacio, el único gasto de memoria es causado por la lista donde vamos anotando las `decisiones` de la partida para mostrarlas en la terminal. Como en total se retiran exactamente n monedas, esta lista tendrá como máximo una longitud de n elementos.

Por lo tanto, la complejidad temporal de la reconstrucción es:

$$\boxed{\mathcal{O}(n)}$$

y su complejidad espacial adicional es:

$$\boxed{\mathcal{O}(n)}$$

3.2.3. Conclusión de costo global

En conclusión, el costo del cálculo iterativo de la matriz de programación dinámica domina el tiempo total del algoritmo. Considerando de manera integral el llenado *Bottom-Up* y la posterior deducción de elecciones, la complejidad temporal total del sistema se mantiene en $\boxed{\mathcal{O}(n^2)}$ y la complejidad espacial total en $\boxed{\mathcal{O}(n^2)}$.

4. Análisis experimental

4.1. Variabilidad de los valores de las monedas

Para estudiar si la distribución de los valores afecta el tiempo de ejecución, se realizaron pruebas con cinco tipos de secuencias, manteniendo fijo el tamaño de entrada en $n = 500$:

- monedas con todos los valores iguales;
- valores aleatorios pequeños;
- valores aleatorios grandes;
- valores en orden creciente;
- valores en orden decreciente.

Para cada tipo de secuencia se realizaron 15 mediciones. Antes de registrar los tiempos se ejecutó una corrida de calentamiento. Para resumir las mediciones se utilizó la mediana de los tiempos, en lugar del promedio, ya que la mediana es menos sensible a mediciones atípicas del entorno de ejecución.

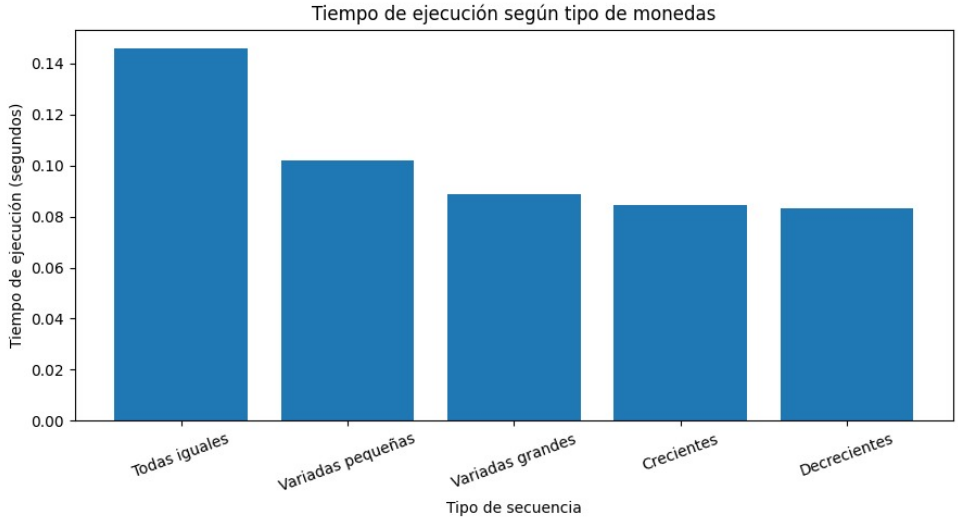


Figura 1: Mediana de los tiempos de ejecución para distintos tipos de secuencias de monedas, con $n = 500$.

Las diferencias observadas entre los conjuntos de datos deben interpretarse como variaciones experimentales. Como el algoritmo procesa todos los intervalos de la entrada, la cantidad de estados depende de n y no de la magnitud ni del orden de los valores. Por ello, para un tamaño fijo, se espera que los tiempos sean del mismo orden, aunque pueden existir pequeñas diferencias debidas al entorno de ejecución y a la medición.

Este comportamiento es compatible con la complejidad temporal teórica $\mathcal{O}(n^2)$. El algoritmo calcula una cantidad cuadrática de subproblemas y realiza una cantidad constante de operaciones por cada uno.

4.2. Medición de tiempos y análisis de complejidad

Para analizar experimentalmente el crecimiento del tiempo de ejecución, se utilizaron los tamaños de entrada $n = 100$, 200, 400, 600, 800 y 1000. Para cada tamaño se generaron monedas

con valores aleatorios entre 1 y 1000. Se realizaron 15 mediciones por tamaño, precedidas por una corrida de calentamiento, y se utilizó la mediana de los tiempos obtenidos.

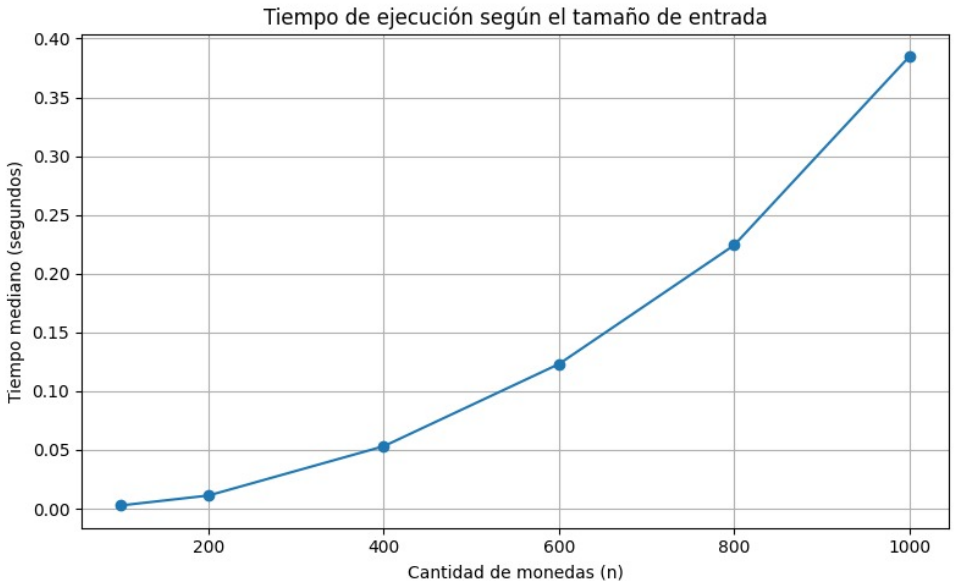


Figura 2: Mediana del tiempo de ejecución del algoritmo según la cantidad de monedas.

Se espera que el tiempo de ejecución aumente al crecer la cantidad de monedas, ya que el número de intervalos que debe resolver el algoritmo es $n(n+1)/2$. En consecuencia, la tendencia esperada es cuadrática. Las mediciones concretas pueden mostrar desviaciones por las condiciones del entorno en el que se ejecutaron las pruebas.

La complejidad temporal teórica es $\mathcal{O}(n^2)$: para cada intervalo $[i, j]$ se realizan solamente una cantidad constante de operaciones. La Figura 2 permite comparar esta predicción con los tiempos observados. Los datos experimentales son evidencia complementaria; la justificación formal de la complejidad proviene del número de estados y del costo constante de cada transición.

5. Conclusiones

Se formuló una solución de programación dinámica para maximizar la ganancia de Sophia frente a un oponente que aplica una estrategia greedy determinista. La recurrencia considera las dos elecciones posibles de Sophia y anticipa la respuesta de Mateo en cada caso. La demostración por inducción establece que los valores almacenados en la matriz son óptimos para todos los intervalos.

La implementación calcula la matriz en tiempo $\mathcal{O}(n^2)$ y espacio $\mathcal{O}(n^2)$, y reconstruye la secuencia de decisiones en tiempo adicional $\mathcal{O}(n)$. Los experimentos permiten observar el comportamiento del algoritmo para distintas distribuciones y tamaños de entrada. En particular, la comparación de estrategias muestra que una elección localmente equivalente puede tener consecuencias futuras diferentes, y que la programación dinámica puede aprovechar esas consecuencias para mejorar la ganancia de Sophia.